Page de garde (non numérotée)

- Titre du projet
- Sous-titre : « Document d'architecture logicielle »
- Version du document
- Date
- Auteurs / équipe projet

1. Contexte et objectifs

1.1 Contexte général

Présenter le problème à résoudre, le domaine métier, la situation actuelle et les limites des solutions existantes.

Expliquer en quelques paragraphes pourquoi le système est nécessaire.

1.2 Objectifs du système

Lister les objectifs principaux du logiciel (3 à 7 points), par exemple : automatiser un processus, améliorer l'accès à l'information, sécuriser des opérations, réduire les erreurs, etc.

1.3 Périmètre de la version

Décrire ce qui est inclus dans cette version (MVP ou première itération) et ce qui est explicitement hors périmètre.

Cette section permet de fixer les limites du projet dès le départ.

2. Exigences fonctionnelles et non fonctionnelles (synthèse)

2.1 Exigences fonctionnelles (vue macro)

Lister les grandes fonctionnalités attendues, sous forme de blocs, sans entrer dans le niveau écran ou API.

Exemples de formulations :

- « Permettre la création, la modification et la suppression de … »
- « Permettre aux utilisateurs de consulter … »
- « Fournir un tableau de bord de … »

2.2 Exigences non fonctionnelles

Décrire les principales contraintes de qualité attendues :

- **Performance** : ordre de grandeur des temps de réponse acceptables, volumes prévus.
- **Sécurité** : besoins en authentification, rôles, protection des données sensibles.
- **Disponibilité et fiabilité** : niveau de disponibilité attendu, tolérance aux pannes.
- **Ergonomie / UX** : simplicité, responsive, accessibilité minimale.

L'objectif est de cadrer les attentes de qualité, sans entrer encore dans les détails techniques.

3. Acteurs et vue d'utilisation

3.1 Acteurs du système

Lister les types d'utilisateurs (ou systèmes externes) qui interagiront avec l'application.

Pour chaque acteur :

- une phrase sur son rôle,
- une phrase sur ce qu'il vient faire dans le système.

3.2 Scénarios d'usage principaux

Décrire quelques scénarios représentatifs (3 à 6), du point de vue des utilisateurs.

Chaque scénario raconte, en quelques étapes, comment un acteur utilise le système pour atteindre un objectif (par exemple : créer un enregistrement, effectuer une transaction, consulter un rapport).

3.3 User stories (optionnel)

Si le projet utilise une approche agile, cette section peut lister des user stories sous la forme :

« En tant que [acteur], je veux [action], afin de [bénéfice]. »

4. Architecture globale du système

4.1 Vue d'ensemble

Présenter, sous forme de texte et idéalement d'un schéma simple, les grands blocs qui composent la solution :

- Interfaces (web, mobile, API publique, etc.)
- Backend ou services applicatifs
- Base(s) de données et stockage de fichiers
- Services ou systèmes externes (paiement, messagerie, autres APIs…)

4.2 Principes d'architecture

Préciser les choix structurants :

- Style architectural : monolithe, microservices, architecture en couches, etc.
- Types de communication : API REST, messages asynchrones, événements, etc.
- Principes généraux : séparation claire entre présentation, logique métier et persistance, par exemple.

Cette section donne une vision macro avant d'entrer dans le détail des modules.

5. Conception des modules (section centrale)

Cette section est le cœur du document : elle décrit la découpe du système en modules, leurs responsabilités, les entités qu'ils portent, leurs attributs et leurs interactions.

5.1 Vue d'ensemble des modules

- Liste des modules avec une phrase de description chacun.
- Schéma (boîtes + flèches) montrant les modules et leurs relations principales.

Exemples de familles de modules (génériques) : gestion des comptes, gestion du catalogue, gestion des opérations métier, reporting, etc.

Les noms et rôles exacts dépendent du projet.

5.2 Description détaillée des modules

Pour chaque module, une sous-section spécifique est définie, en suivant systématiquement le gabarit ci-dessous.

Les **entités et leurs attributs** sont décrits entièrement **dans cette sous-section**.

5.2.x [Nom du module]

1. **Rôle et responsabilité du module**
   - Description claire du rôle global du module : ce qu'il garantit au reste du système.
   - Focus sur la responsabilité métier (ce que le module gère ou protège).
2. **Fonctionnalités prises en charge par le module**
   - Liste structurée des fonctionnalités principales couvertes par ce module :
     - opérations de création / modification / consultation / suppression ;
     - traitements métier spécifiques ;
     - règles de validation importantes, etc.
3. **Entités du module (avec attributs et explications)**

   Cette section décrit toutes les entités **dont le module est responsable**, en détaillant leurs attributs et leur signification.

   Pour chaque entité gérée par le module :

   Nom de l'entité

   - Description générale (rôle de l'entité dans le domaine).
   - Tableau ou liste d'attributs, par exemple :

Attribut	Type / Format	Description métier	Contraintes principales
id	Identifiant (ex : UUID, entier…)	Identifiant unique de l'entité	Unique, obligatoire
nom	Texte	Libellé ou nom lisible par les utilisateurs	Obligatoire, longueur max, unicité éventuelle
statut	Enum / texte	État courant de l'entité (ex. actif, archivé, etc.)	Valeurs autorisées, valeur par défaut
créé_le	Date/heure	Date de création	Renseigné automatiquement

3. Les attributs sont ainsi définis **ici**, à l'intérieur du module, et non dans une section séparée.

   Chaque entité est donc contextualisée dans le module qui la porte.

4. **Interfaces exposées par le module**
   - Liste des interfaces par lesquelles le module est utilisé :
     - API (endpoints principaux, noms et finalités, sans forcément détailler tous les champs) ;
     - écrans ou composants d'interface (si le module a une composante UI) ;

        - événements émis (dans le cas d'une architecture événementielle).
    - Indication du type d'accès (lecture, écriture, modification, etc.).
5. **Flux fonctionnels principaux impliquant le module**
    - Description textuelle de quelques flux clés où le module joue un rôle important.
    - Le cas échéant, diagrammes simples (séquences, activités) pour illustrer un flux critique.
6. **Dépendances vers d'autres modules**
    - Liste des modules dont ce module dépend directement.
    - Description synthétique de l'usage de chaque dépendance (ex. : récupération d'informations, délégation d'une partie du traitement).
7. **Règles métier et contraintes spécifiques au module**
    - Énumération des règles métier majeures propres au module (contrôles, calculs, états).
    - Mention éventuelle de contraintes non fonctionnelles spécifiques à ce module (performance locale, sécurité accrue, etc.).

6. Vue globale du modèle d'information (sans redécrire les attributs) (Pas obligatoire)

Cette section est complémentaire à la section 5 : les entités et leurs attributs ont déjà été définis, module par module.

6.1 Inventaire des entités par module

Récapitulatif synthétique sous forme de tableau, par exemple :

Module	Entités principales gérées
Module A	Entité1, Entité2, …
Module B	Entité3, Entité4, …

Cette section ne redétaille **pas** les attributs, mais donne une vue consolidée.

6.2 Relations entre entités

Schéma global des relations entre entités (diagramme conceptuel ou de classes simplifié), en s'appuyant sur les entités déjà définies dans les modules.

L'objectif est d'offrir une vue d'ensemble du modèle de données, sans dupliquer la description des attributs.

7. Architecture applicative et technique

7.1 Couches logicielles

Décrire l'organisation en couches (par exemple : présentation, application, domaine, infrastructure) et expliquer le rôle de chacune.

7.2 Technologies et frameworks

Lister les technologies retenues pour :

- l'interface utilisateur,
- le backend,
- la base de données,
- les mécanismes de communication,
- les services externes (messagerie, paiement, stockage, etc.).

Cette section doit rester descriptive et justifier brièvement les choix majeurs si nécessaire.

---

8. Sécurité et qualités techniques

8.1 Sécurité

Décrire les mécanismes de sécurité prévus :

- Authentification et gestion des rôles / permissions
- Protection des données sensibles (en transit et éventuellement au repos)
- Journalisation des actions sensibles (audit)

8.2 Autres qualités techniques

Préciser les points importants liés aux exigences non fonctionnelles, par exemple :

- Stratégies de performance (caches, pagination, optimisation des requêtes)
- Stratégies de robustesse (gestion des erreurs, reprises, timeouts)
- Observabilité de base (logs, métriques simples)

---

9. Déploiement et environnements

9.1 Environnements prévus

Indiquer les environnements : développement, test / préproduction, production (ou démonstration pour un projet académique), ainsi que leur usage principal.

9.2 Vue de déploiement

Décrire, sous forme de texte et éventuellement d'un schéma, comment les composants sont déployés :

- serveurs applicatifs,
- base(s) de données,
- services externes critiques,
- éventuels composants réseau.

---

10. Organisation du développement et gestion du code

10.1 Gestion du code source

Préciser les outils utilisés (Git, plateforme d'hébergement de code) et la structure générale du dépôt (répertoires principaux, séparation frontend/backend, documentation).

10.2 Stratégie de branches et contributions

Décrire de manière simple :

- les branches principales (par exemple : main, develop)
- l'utilisation de branches de fonctionnalité (feature branches)
- le processus de contribution (pull requests, revues de code)

11. Risques, limites et évolutions

11.1 Risques principaux

Lister les risques techniques ou organisationnels identifiés (ex. dépendance à un service tiers, complexité de certaines intégrations, ressources limitées).

11.2 Limites connues de la version actuelle

Expliquer clairement ce que le système ne couvre pas encore, ou ce qui est traité de manière simplifiée.

11.3 Pistes d’évolution

Proposer des directions possibles pour les prochaines versions : nouvelles fonctionnalités, enrichissement des modules existants, améliorations techniques, montée en charge, etc.

12. Annexes (facultatif)

- Diagrammes de séquence détaillés
- Spécifications plus complètes d’API
- Gabarits d’ADR (Architecture Decision Records)
- Maquettes d’interface, exemples de payloads JSON, etc.